

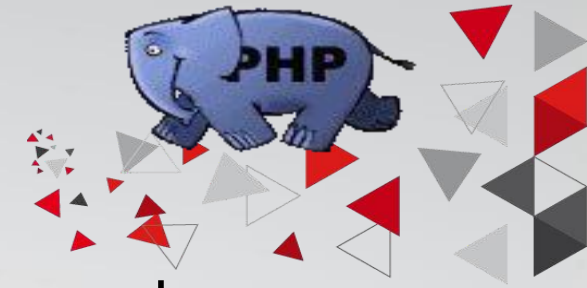


Course 2 – PHP

Forms on PHP

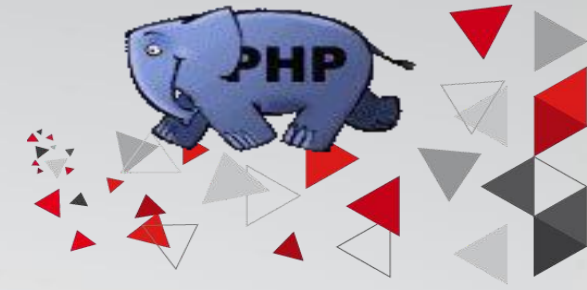
2025 – 2026

Course outcomes



- Understanding HTML Forms: Learn how to create HTML forms and embed them in a web page.
- Handling Form Data: Learn how to retrieve and process data submitted by users via forms.
- Using GET and POST Methods: Understand the differences between GET and POST methods, and when to use each.
- Retrieve Data with PHP: Learn how to use superglobal variables like `$_GET`, `$_POST`, and `$_FILES` to retrieve form data.
- Handling File Submissions: Learn how to handle file submissions via a form, including validating file size and extension.

Useful functions(2/3)



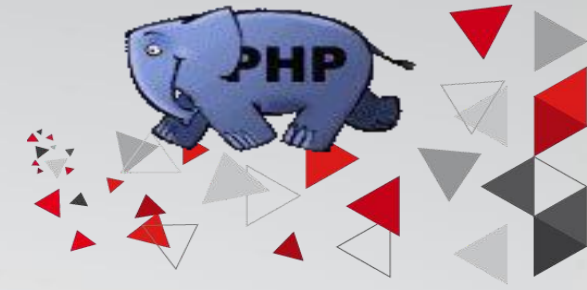
isset()

isset(\$nom) allows you to know if the variable \$name exists and initialized or not, isset returns **TRUE** if the variable is defined and **FALSE** otherwise.

Empty()

Empty(\$nom) determines if a variable contains a non-zero value. Empty returns the value **FALSE** if the variable name is assigned or has a value other than 0 and **TRUE** in other cases.

Useful functions (3/3)



unset()

unset(\$nom) destroy a variable

```
//destroy a variable  
unset($x);  
//destroy an item feom an array  
unset($t[3]);  
//destroy multiple variables  
unset($x,$xy,$z);
```

var_dump

It displays the content of a variable as well as its type.

Form Definition (1/2)



- A form is a very suitable way to create attractiveness between the visitor and the website.
- PHP allows you to process data entered using an HTML form if the ACTION field of the form designates a PHP resource on the server.
- The information entered by the Internet user is retrieved and sent for processing.
- We define a form with the form tag:

```
<form method="GET/POST" action="targetfile">  
</form>
```

Form Definition (2/2)



- The “*method*” attribute defines the method used to send the data: *GET* or *POST*
- By default the value is *GET*
- The *action* attribute defines the file to which the entered information is sent
- By default the value of the “*action*” attribute is the page itself (the data is returned to the same page)

```
<form method="GET/POST" action="fichierCible">  
</form>
```



Form with GET method

- The GET method involves passing data through the URL.
- This aspect deserves special attention because data security is threatened (information visible in the URL).
- You must then try to hide the transmitted values using for example encryption.

Form with POST method



- Another method for submitting forms is the POST method.
- This method should be used in certain cases such as sending files, sending heavy data, sending confidential data.
- The POST method is also recommended if the form triggers a specific action that needs to be repeated each time.
- With the POST method, it is possible to go back and resubmit the form.



Data Recovery(1/4)

- The transmitted data is sent in arrays variables called ***superglobals***.
- ***Superglobals*** are variables that are always accessible, regardless of scope. We can access it from any function, class or file without any special treatment.
- Examples: `$_GET`, `$_POST`, `$_FILES`, `$_SESSION`



Data recovery(2/4)

- The `$_GET` array allows you to retrieve all the data sent by the GET method. It contains all the data transmitted via the URL.
- The `$_POST` array contains the data sent by the POST method.
- The `$_FILES` array is specific to sending files. It contains file information when the user uploads a file via a form.



Data Recovery(3/4)

- To retrieve data, we use the name of the field (the value of the name attribute of the form element) as an index of the `$_POST` array (if the POST method is used) or of the `$_GET` array (if the method GET is used)

- Exemple:

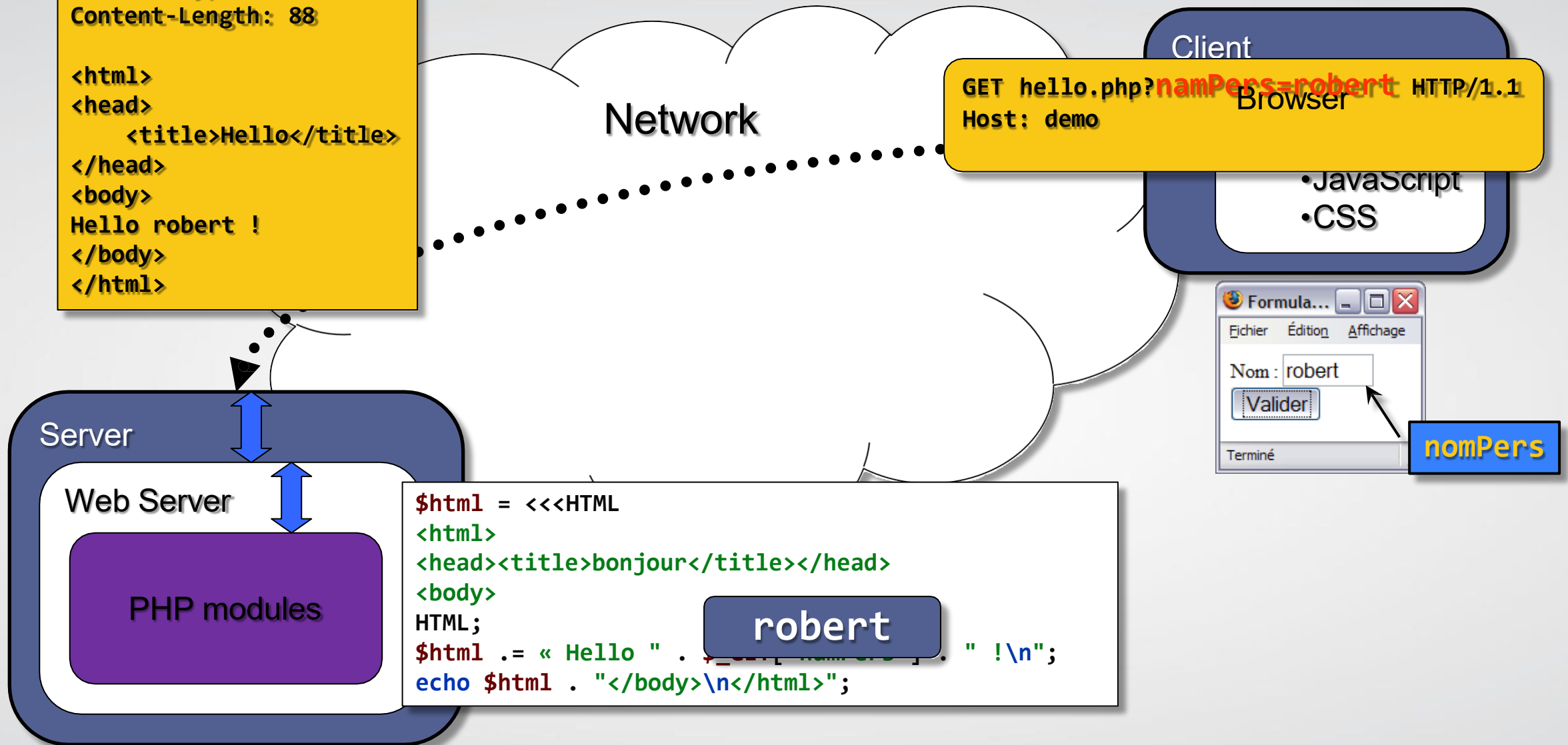
```
$name=$_POST[ 'name' ];
```

Or

```
$name=$_GET [ 'name' ];
```



Form Data Processing



Example:

form.html



```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title> Document </title>
  </head>
  <body>
    <form method="get" action="valid.php">
      Name:<input type="text" name="namePers">
        <input type="submit" value="Send">
    </form>
  </body>
</html>
```

Example:



valid.php

```
<?php
if(isset($_GET['namePers']))
{
    if(!empty($_GET['namePers']))
    {
        echo "you have entered ".$_GET['namePers']."\n";
    }
    else
    {
        echo "No value entered";
    }
}
else
{
    echo "incorrect use";
}
?>
```



Forms containing « Select »

- We use the tag `<select>` to which we give a name (in the example: *“choice”*).
- Here, a variable `$_GET[ 'choice' ]` will be created, and it will contain the choice made by the user.

form.html

```
<select name="choice[]">
  <option value="">-- Please choose an option</option>
  <option value="strawberry">Strawberry</option>
  <option value="banana">Banana</option>
  <option value="cherry">Cherry</option>
  <option value="apple">Apple</option>
</select>
```



Forms containing « Select »

valid.php

```
if(isset($_GET['choice']) && !empty($_GET['choice'])&& $_GET['choice'][0] != '')
{
    /* The variable $_GET[choice] is defined and is not empty
    */
    foreach($_GET['choice'] as $fruit)
    {
        echo "<p> You choose ".$fruit;
    }
}
else
{
    echo "<p>You didn't choose any fruit";
}
```




Forms containing « Chechbox»

form.html

```
<input type="checkbox" name="case[]" id="case" value="case 1"/>
<label for="case"> Case 1</label> <br>
<input type="checkbox" name="case[]" id="case" value="case 2"/>
<label for="case"> Case 2</label><br>
<input type="checkbox" name="case[]" id="case" value="case 3"/>
<label for="case"> Case 3</label>
```



Forms containing « Checkbox »

valid.php

```
if(isset($_GET['case']))
{
    foreach($_GET['case'] as $valeur)
    {
        echo "<br>".$valeur . "<br>";
    }
}
```



Forms containing « Radio »

form.html

```
<input type="radio" name="gender" value="women"> Women  
<br>  
<input type="radio" name="gender" value="man"> Man  
<br>
```



Forms containing « Radio »

valid.php

```
echo "You are: <b>" . $_GET[ 'gender' ] . "</b><br>";
```

File submission form



- As soon as your form asks visitors to send a file, you must add the `enctype="multipart/form-data"` attribute to the tag `<form>`.
- Thanks to `enctype`, the visitor's browser knows that it is about to send files.
- The tag used to send a file is the `<input type='file'>` tag.
- You must give a name to this form field (using the `name` attribute) so that PHP can recognize the field subsequently.

File submission form



form.html

```
<label for="screenshot" class="form-label" > Your Screenshot</label>  
<input type="file" class="form-control" id="screenshot" name="screenshot"/>
```

File submission form



- For each file sent, a variable `$_FILES['field_name']` is created.
- In our case, the variable will be called `$_FILES['screenshot']`
- This variable is an array that contains several pieces of information about the file:

File submission form



Variable	Meaning
<code>\$_FILES['screenshot']['name']</code>	Contains the name of the file sent by the visitor.
<code>\$_FILES['screenshot']['type']</code>	Indicates the type of file sent. If it is a gif image for example, the type will be image/gif
<code>\$_FILES['screenshot']['size']</code>	Indicates the size of the sent file. Please note: this size is in bytes. It takes about 1,000 bytes to make 1 KB, and 1,000,000 bytes to make 1 MB. The upload size is limited by PHP. By default, it is impossible to upload files larger than 8 MB.
<code>\$_FILES['screenshot']['tmp_name']</code>	After sending, the file is placed in a temporary directory on the server while waiting for your PHP script to decide whether or not to store it for good. This variable contains the temporary location of the file (PHP handles this).
<code>\$_FILES['screenshot']['error']</code>	Contains an error code allowing you to know if the sending was successful or if there was a problem and if so, what. The variable is 0 if there was no error.

File submission form



1) *Test sending a file:*

- To check that a file has been sent, you must test if the `$_FILES[ 'screenshot' ]` variable exists with `isset()`.
- You must also check that there are **no** sending errors.

valid.php

```
if(isset($_FILES[ 'screenshot' ]) && $_FILES[ 'screenshot' ][ 'error' ]==0)
{
    echo "File sent";
}
else
{
    echo "You haven't chosen any file";
}
```

File submission form



2) Checking file size

- To prevent the file from exceeding 1 MB, or approximately 1,000,000 bytes.
We must therefore test `$_FILES['screenshot']['size']`

valid.php

```
if(isset($_FILES['screenshot']) && $_FILES['screenshot']['error']==0)
{
    //test if the file is not too big
    if($_FILES['screenshot']['size'] <= 1000000)
    {

    }
}
```

File submission form



3) *Checking the extension*

- To retrieve the file extension in a variable using this code:

```
$fileInfo = pathinfo($_FILES['screenshot']['name']);  
$extension = $fileInfo['extension'];
```

File submission form



3) *Checking the extension*

- The `pathinfo` function returns an array containing, among other things, the file extension in `$fileInfo['extension']`.
- We store this in a `$extension` variable.
- Once the extension has been retrieved, we can compare it to an array of authorized extensions, and check whether the retrieved extension is one of the authorized extensions using the `in_array()` function..

```
$fileInfo = pathinfo($_FILES['screenshot']['name']);  
$extension = $fileInfo['extension'];
```

File submission form

3) Checking the extension

```
if(isset($_FILES['screenshot']) AND $_FILES['screenshot']['error']==0)
{
    //test if the file is not too big
    if($_FILES['screenshot']['size'] <= 1000000)
    { //Test if the extension is authorized
        $fileInfo=pathinfo($_FILES['screenshot']['name']);
        $extension=$fileInfo['extension'];
        $allowedExtensions=['jpg','jpeg','gif','png'];
        if(in_array($extension,$allowedExtensions))
        {

        }
    }
}
```

File submission form



4) *Validation of file upload*

If everything is good, we accept the file by calling `move_uploaded_file()`

This function takes two parameters:

1. The temporary name of the file (we have it with `$_FILES['screenshot']['tmp_name']`).
2. The path which is the name under which the file will be permanently stored.

We can use the original file name `$_FILES['screenshot']['name']` or generate a name randomly.

File submission form



4) Validation of file upload

```
if(isset($_FILES['screenshot']) AND $_FILES['screenshot']['error']==0)
{
    //test if the file is not too big
    if($_FILES['screenshot']['size'] <=
    1000000)
    { //Test if the extension is authorized
        $fileInfo=pathinfo($_FILES['screenshot']['name']);
        $extension=$fileInfo['extension'];
        $allowedExtensions=['jpg','jpeg','gif','png'];
        if(in_array($extension,$allowedExtensions))
        {
            //we can validate the file and store it permanently
            move_uploaded_file($_FILES['screenshot']['tmp_name'],'uploads/'.basename($_FILES['screenshot']['name']));
            echo "the shipment was successful!";
        }
    }
}
```

File submission form



4) Validation of file upload

- In this example, the file is placed in an “Uploads” subfolder. We will keep the same file name as the original one.
- As `$_FILES['screenshot']['name']` contains the entire path to the original file (`C:\folder\file.png` , for example), we will need to extract the file name.
- To do this, we can use the `basename` function which will just return “file.png”.



References

<https://www.php.net/manual/fr/>

https://www.w3schools.com/php/php_forms.asp

<https://www.geeksforgeeks.org/php-form-processing/>

<https://www.php.net/manual/fr/features.file-upload.php>

https://www.w3schools.com/php/php_file_upload.asp